Adapta

General Purpose Transfer Learning Framework for Cross-Domain Classification

Technical Report

SE 801: Software Project Lab III

Supervised by:

Dr. Mohammad Shoyaib

Professor

Institute of Information Technology

University of Dhaka

Submitted by:

Hasnain Sheikh

Roll: BSSE-1408

Institute of Information Technology

University of Dhaka

Submitted to:

SPL 3 Program Committee

Submission Date: 02 August, 2026

Institute of Information Technology

University of Dhaka

Letter of Transmittal

02 August, 2026

BSSE 4th Year Exam Committee

Institute of Information Technology

University of Dhaka

Subject: Technical Report Submission – General Purpose Transfer Learning Framework for Cross-Domain Classification.

Dear Sir / Madam,

I am pleased to submit the technical report of my Software Project Lab III on the project titled "Adapta: General Purpose Transfer Learning Framework for Cross-Domain Classification". This report documents the requirement analysis, system modeling, dataset and information modeling, software design, architectural design, and the preliminary test plan of the proposed framework.

I have tried my best to present the technical details of the project clearly and accurately. However, the work may still lack perfection in places. I would therefore be grateful if you would kindly accept my report and provide any feedback that could help me improve it further.

Thank you for your consideration.

Sincerely,

Hasnain Sheikh
Roll: BSSE-1408
Institute of Information Technology
University of Dhaka

Acknowledgement

I am sincerely grateful to the Institute of Information Technology, University of Dhaka, for giving me the opportunity to carry out my Software Project Lab III on the project titled "Adapta: General Purpose Transfer Learning Framework for Cross-Domain Classification". This project has given me the chance to apply theoretical knowledge of machine learning and software engineering to a practical research problem.

I would like to express my heartfelt gratitude to my respected supervisor, Dr. Mohammad Shoyaib, Professor, Institute of Information Technology, University of Dhaka, for his continuous guidance, constructive criticism, and encouragement throughout every phase of this work. His insights into transfer learning, domain adaptation, and research methodology have been instrumental in shaping the direction, design, and scope of this framework.

I am also thankful to my classmates for their cooperation, helpful discussions, and suggestions during different phases of the project. Finally, I acknowledge the resources and academic environment provided by the Institute of Information Technology, which made the successful progress of this project possible.

Table of Contents

Letter of Transmittal	2
Acknowledgement	3
Abstract	4
1. Introduction	7
1.1 Purpose	7
1.2 Motivation	8
1.3 Problem Statement	9
1.4 Objectives	9
1.5 Scope	10
1.6 Deliverables	10
1.7 Purpose of this Document	11
2. Project Description and Requirements Analysis	12
2.1 Normal Requirements	12
2.2 Expected Requirements	13
2.3 Exciting Requirements	14
2.4 Non-Functional Requirements	14
2.5 Stakeholders	15
2.6 Usage Scenario	15
3. Scenario-Based Modeling	18
3.1 Use Case Modeling	18
3.1.1 Level 0 Use Case Diagram	19
3.1.2 Level 1 Use Case Diagram	20
3.1.2.1 Authentication	20
3.1.2.2 Upload Datasets	21

3.1.2.3 Data Preprocessing & Transfer Learning Method Selection 21

3.1.2.4 Target Level Prediction 21

3.1.2.6 Report Generation 22

3.1.2.7 Store Results 22

3.1.3 Level 1.1 Authentication 22

3.1.4 Level 1.2 Upload Dataset 23

3.1.5 Level 1.3 Data Preprocessing 23

3.1.6 Level 1.4 Label Prediction 24

3.2 Swimlane Diagram 25

4. Data-Based Modeling 27

4.1 Data Object Identification 27

4.2 Principal Data Objects 27

4.3 Entity Relationship Diagram 28

5. Class-Based Modeling 28

5.1 Analysis Classes 28

5.2 Class Cards 29

5.3 Class Responsibility and Collaboration Diagram 34

6. Software Design and Architectural Design 35

6.1 Architectural Context Diagram 35

6.2 Top-Level Components 36

6.3 Mapping Requirements to Software Architecture 36

7. Preliminary Test Plan 38

7.1 Testing Objectives 38

7.2 Test Cases 38

8. Timeline 43

Abstract

Conventional machine learning models assume that training and test data are drawn from the same distribution and require a large volume of labeled data. In many practical settings this assumption does not hold: labeled data in the target domain is scarce, expensive to annotate, or unavailable, while abundant labeled data exists in a related but differently distributed source domain. Transfer learning and domain adaptation address this gap by transferring knowledge across domains, but they introduce their own difficulties, most notably negative transfer, in which knowledge from a poorly related source domain degrades target-domain performance.

This project proposes a General Purpose Transfer Learning Framework, a modular desktop application that unifies dataset management, preprocessing, domain adaptation, classifier training, evaluation, and visualization for cross-domain classification tasks over text and image data. The framework implements a common interface over a set of classical and recent domain adaptation methods, namely Transfer Component Analysis (TCA), Joint Distribution Adaptation (JDA), Balanced Distribution Adaptation (BDA), Correlation Alignment (CORAL), Geodesic Flow Kernel (GFK), Joint Clustering and Subspace Alignment (JCSA), Domain Adaptation through Locality Information (DALI), Orthogonal Core-extraction of Matrices TransferLearning (OCMTL), and the proposed enhanced method. A researcher can load a source and a target dataset, configure preprocessing and hyperparameters such as latent feature dimension, kernel type, and regularization, select a baseline classifier, and execute an adaptation experiment from a single interface.

After execution, the framework reports Accuracy, Precision, Recall, F1-score, AUC, and Maximum Mean Discrepancy (MMD), and renders the feature distributions before and after adaptation using PCA and t-SNE projections. A cross-domain transfer analyzer quantifies domain similarity, flags likely negative transfer, and recommends suitable adaptation methods for a given domain pair. Experiment configurations can be saved and reloaded for reproducibility, and results, plots, and comparison tables can be exported as PDF or CSV.

The expected outcome is a reusable, extensible research and teaching platform that lowers the effort required to run reproducible cross-domain experiments, makes comparative evaluation of adaptation methods straightforward, and provides interpretable evidence of when transfer helps and when it harms.

1. Introduction

Machine learning has become the standard approach for classification problems in text, vision, and software analytics. The effectiveness of a conventional supervised model, however, rests on two assumptions: that a sufficient quantity of labeled training data is available, and that the training and test data are drawn from the same underlying distribution. In real deployments both assumptions are frequently violated. Annotating data for a new domain is expensive and slow, and the data that is already available typically comes from a related but distributionally different source, such as product reviews from a different product category, images captured with a different camera under different lighting, or defect records from a different software project.

Transfer learning addresses this situation by reusing knowledge acquired in a label-rich source domain to improve learning in a label-scarce target domain. Domain adaptation, the branch of transfer learning most relevant to this project, seeks a feature representation or a transformation under which the source and target distributions become close enough that a classifier trained on source data generalizes to target data. A substantial body of methods has been proposed for this purpose, each with different assumptions about which discrepancy is being minimized, different computational costs, and different hyperparameters.

In practice, a researcher who wishes to compare these methods faces a considerable engineering burden. Reference implementations are scattered across MATLAB and Python, differ in their input formats and preprocessing conventions, expose incompatible interfaces, and report different evaluation metrics. Consequently, comparing methods fairly on a new dataset consumes far more effort than the scientific question itself warrants, and results are difficult to reproduce.

The General Purpose Transfer Learning Framework proposed in this project is a modular desktop application that removes this burden. It provides one pipeline for loading datasets, tagging source and target domains, applying preprocessing, executing any of the supported adaptation methods, training a baseline classifier on the adapted representation, evaluating performance on the target domain, and visualizing the resulting feature distributions. It additionally analyzes domain similarity, warns about likely negative transfer, and exports reproducible experiment records.

1.1 Purpose

The primary purpose of this framework is to provide a single, reusable, and extensible platform on which cross-domain classification experiments can be designed, executed, compared, and reproduced. Specifically, the framework aims to:

- Unify data loading, preprocessing, adaptation, training, and evaluation into one configurable pipeline so that experiments differ only in the parameters the researcher intends to vary.
- Offer a common programmatic and graphical interface over multiple transfer learning and domain adaptation methods, allowing fair like-for-like comparison on the same data and the same evaluation protocol.

- Quantify the relationship between a source and a target domain, using Maximum Mean Discrepancy and feature distribution overlap, so that the suitability of transfer can be assessed before and after adaptation.
- Detect and report negative transfer, that is, cases in which adaptation degrades target performance relative to a no-transfer baseline.
- Make the effect of adaptation interpretable through PCA and t-SNE visualizations of the source and target feature spaces before and after alignment.
- Support reproducibility by saving experiment configurations and exporting results, plots, and comparative reports in PDF and CSV form.
- Provide a lowered-effort entry point for students and teachers who want to study or demonstrate transfer learning without first re-implementing the underlying methods.

1.2 Motivation

The motivation for this project arises from three observed difficulties.

Annotation scarcity. Supervised learning in a new domain requires labeled examples that are often unavailable. Manual annotation is costly in time and expertise, and for some domains, such as medical imagery or defect-prone software modules, competent annotators are rare. Transfer learning offers a route to usable models without full annotation of the target domain.

Domain mismatch. Even when data is plentiful, distribution shift between the data a model was trained on and the data it is applied to degrades accuracy sharply. Marginal shift, conditional shift, and their combination each require different treatment, and no single adaptation method dominates across all shift types and datasets.

Lack of a unified experimental framework. Existing transfer learning methods are typically implemented as separate tools or codebases with different preprocessing pipelines, parameter settings, and evaluation procedures. This makes fair comparison difficult and often causes differences in experimental setup to influence the results. A unified framework that supports multiple transfer learning methods under a single interface, using the same preprocessing pipeline, evaluation metrics, and experimental protocol, enables consistent benchmarking, simplifies experimentation, and improves reproducibility.

Negative transfer. Transfer is not unconditionally beneficial. When source and target domains are weakly related, transferring knowledge can actively harm target performance. Existing tooling gives the researcher little support for anticipating this outcome, so it is usually discovered only after a full and possibly expensive experiment has been run.

These difficulties are compounded by the fragmented state of available implementations. A framework that standardizes the pipeline, supports several adaptation methods behind one interface, and explicitly measures domain relatedness therefore has clear value both as a research instrument and as a teaching aid.

1.3 Problem Statement

Cross-domain classification suffers when the target domain has few or no labels and its distribution differs from that of the available source data. Although many domain adaptation methods have been proposed to correct this shift, there is no unified, extensible software platform through which these methods can be applied to arbitrary datasets, configured consistently, evaluated under an identical protocol, and compared against one another and against a no-transfer baseline. As a result, method selection for a new domain pair is largely guesswork, negative transfer is detected only retrospectively, and published comparisons are difficult to reproduce.

This project therefore addresses the following problem: how to design and implement a modular, general-purpose framework that (i) accepts heterogeneous software, text and image datasets, (ii) applies a configurable set of transfer learning and domain adaptation methods through a common interface, (iii) evaluates target-domain performance with a standard metric suite, (iv) quantifies domain similarity and identifies negative transfer, and (v) presents the results in an interpretable, exportable, and reproducible form.

1.4 Objectives

The objectives of the project are as follows:

1. To design a unified data processing pipeline supporting dataset loading, validation, normalization, standardization, feature preparation, and source/target domain tagging.
2. To implement a modular adaptation layer exposing TCA, JDA, BDA, CORAL, GFK, JCSA, DALI, and OCMTL through a single abstract interface, such that a new method can be added without modifying the rest of the system.
3. To design and implement the proposed method, as the contribution of this project, and to evaluate it against the existing methods under identical conditions.
4. To provide baseline classifier training on the adapted feature space and a no-transfer baseline for reference.
5. To implement an evaluation module computing Accuracy, Precision, Recall, F1-score, AUC, and MMD, together with comparative tabulation across methods.
6. To implement a cross-domain transfer analyzer that measures domain similarity, detects negative transfer, and recommends candidate adaptation methods.
7. To implement visualization of feature distributions before and after adaptation using PCA and t-SNE, along with domain alignment scatter plots.
8. To build an interactive PyQt-based desktop interface for dataset upload, experiment configuration, real-time progress monitoring, and result inspection.
9. To support saving and reloading of experiment configurations, and export of results and visualizations in PDF and CSV format.

1.5 Scope

The scope of the framework is defined below.

1.5.1 In Scope

- A desktop application implemented in Python with a PyQt graphical interface.
- Support for tabular feature representations of text and image datasets in .csv, .mat, and .arff formats.
- Explicit source-domain and target-domain tagging, with support for multiple domain pairs from the same dataset family.
- Configurable preprocessing: normalization, standardization, and feature preparation, together with train/test splitting.
- Shallow, feature-transformation based domain adaptation methods: TCA, JDA, BDA, CORAL, GFK, JCSA, DALI, OCMTL, and the proposed method.
- Baseline classifiers available through scikit-learn, such as k-nearest neighbours, support vector machines, and logistic regression.
- Evaluation using Accuracy, Precision, Recall, F1-score, AUC, and MMD, with comparative analysis across methods.
- Domain similarity analysis, negative transfer detection, and method recommendation.
- PCA and t-SNE visualization of feature spaces before and after adaptation.
- Experiment configuration persistence and report export in PDF and CSV.

1.5.2 Out of Scope

- End-to-end deep domain adaptation architectures require GPU training, such as adversarial adaptation networks; the framework consumes pre-extracted deep features, for example DeCAF, rather than training feature extractors itself.
- Raw image or raw text ingestion with in-framework feature extraction pipelines; datasets are expected in feature-vector form.
- Regression, clustering, and sequence-labelling tasks; the framework targets classification only.
- Distributed or cluster-scale training.

1.6 Deliverables

The following artifacts are to be delivered at the end of the project:

- A working desktop application implementing the complete experiment pipeline described above.
- A modular Python library containing the implemented adaptation methods behind a common interface, usable independently of the graphical interface.

- An implementation of the proposed method together with its experimental evaluation against the baseline methods.
- A benchmark result set covering the supported datasets, including comparative accuracy tables and MMD measurements.
- This technical report, covering requirement analysis, system modeling, design, architecture, AI engineering design, and the test plan.
- Source code with version history maintained on GitHub, together with a user guide describing installation, dataset preparation, and experiment execution.

1.7 Purpose of this Document

The purposes of this document are:

- To identify, analyze, and record the functional and non-functional requirements of the framework.
- To model the system through use case, activity, data, and class-based modeling.
- To describe the software design and architecture, and to establish traceability from requirements to architectural components.
- To document the AI engineering design, including the pipeline, method selection rationale, and hyperparameter configuration.
- To define the preliminary test plan, testing objectives, and the features to be tested.
- To present the project timeline and current progress for midterm evaluation.
- To serve as a reference for supervisors, evaluators, and any future developer extending the framework.

2. Project Description and Requirements Analysis

The General Purpose Transfer Learning Framework is a modular desktop application for cross-domain classification. A researcher imports a source dataset and a target dataset, inspects their properties, applies preprocessing, chooses an adaptation method and a baseline classifier, sets the relevant hyperparameters, and runs the experiment. The framework performs domain adaptation, trains the classifier on the transformed feature space, evaluates it on the target domain, and reports both quantitative metrics and visual evidence of how the two distributions were aligned. Experiment configurations are persisted so that any reported result can be regenerated.

Requirements are grouped following the Quality Function Deployment convention into normal, expected, and exciting requirements, and are then complemented by the non-functional requirements and the stakeholder analysis.

2.1 Normal Requirements

Normal requirements are the explicitly stated capabilities without which the framework does not fulfil its stated goal. Their presence produces satisfaction proportional to how well they are implemented.

- **Dataset import.** The system must allow the user to import datasets from disk in .csv, .mat, and .arff formats.
- **Dataset validation.** The system must verify structural integrity of each imported dataset, checking feature-count consistency, label presence, missing values, and type correctness, and must report any violation to the user rather than failing silently.
- **Dataset property inspection.** The system must display the number of samples, number of features, number of classes, class distribution, and data types of each loaded dataset.
- **Domain assignment.** The system must allow the user to designate one loaded dataset as the source domain and another as the target domain.
- **Preprocessing.** The system must offer configurable preprocessing operations, including min-max normalization, z-score standardization, L2 normalization, and feature preparation, applied consistently to both domains.
- **Transfer method selection.** The system must implement TCA, JDA, BDA, CORAL, GFK, JCSA, DALI, OCMTL, and the proposed method, each selectable from the interface.
- **Baseline classifier training.** The system must allow the user to select a baseline classifier and to train it on the adapted feature representation.
- **Hyperparameter configuration.** The system must allow the user to set method-specific hyperparameters, including latent feature dimension, kernel type, kernel parameter, regularization coefficient, and the number of iterations where applicable.

- **Model evaluation.** The system must evaluate the trained model on the target domain and report Accuracy, Precision, Recall, F1-score, and AUC.
- **Experiment execution.** The system must execute the complete pipeline — preprocessing, adaptation, training, evaluation — as one user-initiated operation and report its outcome.
- **Result display.** The system must present the results of a completed experiment in a readable tabular form within the interface.

2.2 Expected Requirements

Expected requirements are those a user of such a framework assumes will be present even when not stated. Their absence causes dissatisfaction disproportionate to their apparent importance.

- **MMD computation.** The system should compute the Maximum Mean Discrepancy between the source and target domains before and after adaptation, quantifying how much the distribution gap was reduced.
- **Feature space visualization.** The system should project source and target features into two dimensions using PCA and t-SNE and render them before and after adaptation, so the alignment achieved is directly observable.
- **Comparative analysis.** The system should run the same experiment configuration across several adaptation methods and present the results side by side in a single comparison table.
- **Negative transfer detection.** The system should compare adapted performance against a no-transfer baseline and flag the experiment as negative transfer when adaptation reduces target accuracy.
- **Experiment persistence.** The system should save the full configuration of an experiment, including dataset paths, domain assignment, preprocessing choices, method, hyperparameters, and random seed, and reload it to reproduce the result.
- **Report export.** The system should export results, comparison tables, and visualizations as PDF reports and CSV files.
- **Experiment dashboard.** The system should present a dashboard summarizing recent experiments, their configurations, and their headline metrics.
- **Progress feedback.** The system should indicate progress during long-running adaptation and remain responsive rather than freezing the interface.
- **Extensible method registry.** The system should be structured so that a new adaptation method can be added by implementing a single interface, without modification to the data, evaluation, or presentation layers.
- **Error handling.** The system should report actionable messages for malformed datasets, incompatible feature dimensions between domains, and singular or ill-conditioned matrices arising during adaptation.

2.3 Exciting Requirements

Exciting requirements go beyond the user's expectations. Their absence is not penalized, but their presence distinguishes the framework from existing tooling.

- **Transfer method recommendation.** The system may analyze the source-target pair before adaptation and rank the available methods by expected suitability, using domain similarity, feature dimensionality, class overlap, and previously recorded outcomes.
- **Domain similarity scoring.** The system may quantify domain relatedness through a composite score combining MMD, feature distribution overlap, and class-conditional divergence, and present it as a pre-experiment advisory.
- **Interactive alignment visualization.** The system may render an interactive scatter plot in which individual points can be inspected, and source and target subsets toggled.
- **Batch experiment execution.** The system may evaluate all supported methods over all available domain pairs of a dataset family in a single batch and produce a full comparison matrix.
- **Adaptation interpretability.** The system may report the contribution of individual feature dimensions to the alignment achieved, giving insight into what the adaptation actually changed.

2.4 Non-Functional Requirements

The non-functional requirements constrain how the framework must behave rather than what it must do.

ID	Category	Requirement
NFR-1	Performance	A single adaptation run on a benchmark dataset of up to 3,000 samples and 4,096 features must complete within two minutes on a standard laptop with 16 GB RAM. Interface actions must respond within two seconds.
NFR-2	Usability	A user familiar with machine learning but not with this framework must be able to configure and run a complete experiment without consulting external documentation. Parameter fields must carry inline descriptions and sensible defaults.
NFR-3	Reliability	Invalid datasets, incompatible domain dimensions, and numerically unstable adaptation must be caught and reported without terminating the application or losing unsaved configuration.
NFR-4	Reproducibility	Given a saved configuration and a fixed random seed, re-execution must reproduce reported metrics exactly for

ID	Category	Requirement
		deterministic methods, and within a stated tolerance for stochastic components such as t-SNE.
NFR-5	Modularity	Data management, preprocessing, adaptation, training, evaluation, analysis, visualization, and presentation must be separate modules communicating through defined interfaces. Adding a method must not require changes outside the adaptation module.
NFR-6	Maintainability	The codebase must follow PEP 8, carry docstrings on all public interfaces, and be version-controlled on GitHub with meaningful commit history.
NFR-7	Portability	The application must run on Windows and Linux with Python 3.10 or later, without platform-specific dependencies beyond the declared requirements file.
NFR-8	Scalability	Memory use must degrade gracefully with dataset size, and the interface must warn the user before attempting an experiment whose estimated memory demand exceeds available resources.
NFR-9	Correctness	Each implemented adaptation method must reproduce the published accuracy of its original paper on at least one standard benchmark, within a documented tolerance.
NFR-10	Interoperability	Exported CSV must be directly loadable by pandas and standard spreadsheet software; exported PDF must be self-contained.

Table 1: Non-functional requirements

2.5 Stakeholders

The stakeholders of the framework and their respective concerns are listed below.

Stakeholder	Type	Interest and Interaction
Researcher	Primary actor	Uses the framework to run cross-domain experiments, compare adaptation methods, evaluate the proposed method, and produce reproducible results for publication. The most demanding user in terms of configurability and correctness.
Teacher	Primary actor	Uses the framework to demonstrate domain shift, adaptation, and negative transfer in a classroom setting, relying particularly on the visualization and comparison features.

Stakeholder	Type	Interest and Interaction
Student	Primary actor	Uses the framework to learn transfer learning concepts by experimentation, running prepared configurations and observing the effect of parameter changes without needing to implement methods.
Admin	Secondary stakeholder	Evaluates the technical soundness, scope, and progress of the project, and assesses the contribution of the proposed method.

Table 2: Stakeholders of the framework

2.6 Usage Scenario

The usage scenarios below describe how the framework is used in practice and provide the basis for the scenario-based modeling of the next chapter.

2.6.1 Dataset Import and Domain Assignment

The researcher launches the framework and opens the dataset management module. Two datasets are selected from disk, for example the Amazon and Webcam subsets of Office-Caltech in DeCAF feature form. The framework validates each file, confirms that the feature dimensionality matches across the two, and displays sample count, feature count, class count, class distribution, and data types for each. The researcher then tags one dataset as the source domain and the other as the target domain. If the two datasets have incompatible feature dimensions, the framework reports this immediately and blocks the experiment rather than allowing a run that would fail later.

2.6.2 Preprocessing Configuration

The researcher selects the preprocessing operations to apply. Options include min-max normalization, z-score standardization, and L2 normalization, along with the choice of whether to fit the preprocessing statistics on the source domain alone or on the union of both domains. The chosen settings are applied identically to both domains, so that any observed difference in performance is attributable to the adaptation method and not to inconsistent preprocessing.

2.6.3 Method Selection and Hyperparameter Configuration

The researcher chooses an adaptation method from the registry, for example BDA, and a baseline classifier, for example a 1-nearest-neighbour classifier as used in the standard literature protocol. The interface presents only the hyperparameters relevant to the chosen method: for BDA these are the latent feature dimension, kernel type, kernel bandwidth, regularization coefficient, the balance factor between marginal and conditional distributions, and the iteration count. Each field carries a default drawn from the original publication so that a valid experiment can be launched without expert tuning.

2.6.4 Experiment Execution

The researcher starts the experiment. The framework applies preprocessing, executes the adaptation method to obtain a shared latent representation, trains the classifier on the transformed source data, and predicts labels for the transformed target data. Progress is reported at each stage, and the interface remains responsive so the run can be cancelled. Should adaptation fail numerically, for example through a singular matrix, the framework reports the cause and the parameter most likely responsible.

2.6.5 Evaluation and Visualization

On completion, the framework displays Accuracy, Precision, Recall, F1-score, and AUC on the target domain, alongside MMD measured before and after adaptation. The researcher opens the visualization panel and inspects PCA and t-SNE projections of the source and target features, shown side by side for the original and the adapted representation. The degree to which the two domain clouds overlap after adaptation provides an interpretable account of what the method achieved.

2.6.6 Comparative Analysis and Negative Transfer

The researcher then requests a comparative run in which the same configuration is evaluated across several adaptation methods and against the no-transfer baseline. The framework produces a single table ranking the methods by target accuracy. Any method whose adapted accuracy falls below the no-transfer baseline is explicitly flagged as negative transfer, and the domain similarity score is presented alongside so the researcher can relate the outcome to the relatedness of the two domains.

2.6.7 Persistence and Export

Finally, the researcher saves the experiment configuration under a descriptive name for later reuse, and exports the results. A PDF report containing the configuration, metric table, comparison table, and plots is produced for inclusion in a paper or presentation, and a CSV file containing the raw metrics is produced for further analysis.

3. Scenario-Based Modeling

Scenario-based modeling describes the system from the point of view of the actors who use it. It establishes what the system does before any decision is made about how it does it, and it provides the vocabulary used consistently throughout the remainder of this document.

3.1 Use Case Modeling

A use case describes the behaviour of the system as it responds to a request from one of its stakeholders. It records a structured account of how an actor interacts with the system under a defined set of circumstances. A use case diagram presents these accounts visually, together with the actors involved, in a form accessible to both technical and non-technical readers.

The first step in use case modeling is to identify the actors. An actor is any entity that interacts with the system within the scope of the behaviour being described, and represents a role rather than a specific individual.

Primary actors interact with the system directly to obtain its intended benefit. For this framework the primary actors are the Researcher, the Teacher, and the Student. The Researcher exercises the full capability of the system, including method comparison, negative transfer analysis, and export for publication. The Teacher uses the system chiefly for demonstration, relying on visualization and comparison. The Student runs prepared or self-configured experiments for learning purposes. Because their goals differ in depth rather than in kind, the Teacher and Student inherit a subset of the Researcher's interactions.

Secondary actors support the system so that the primary actors can achieve their goals. Here they are the File System, which supplies dataset files and receives exported reports; the Dataset Repository, the external source of the benchmark datasets; and the Computation Libraries — NumPy, SciPy, scikit-learn, Matplotlib and Seaborn — which perform the underlying numerical, learning, and rendering work.

Use Case Diagram

Level 0
Use Case ID: 0
Name: Adapta: General Purpose Transfer Learning Framework
Primary Actor: Researcher, Student, Teacher, ML Engineer
Secondary Actor: Email, Admin

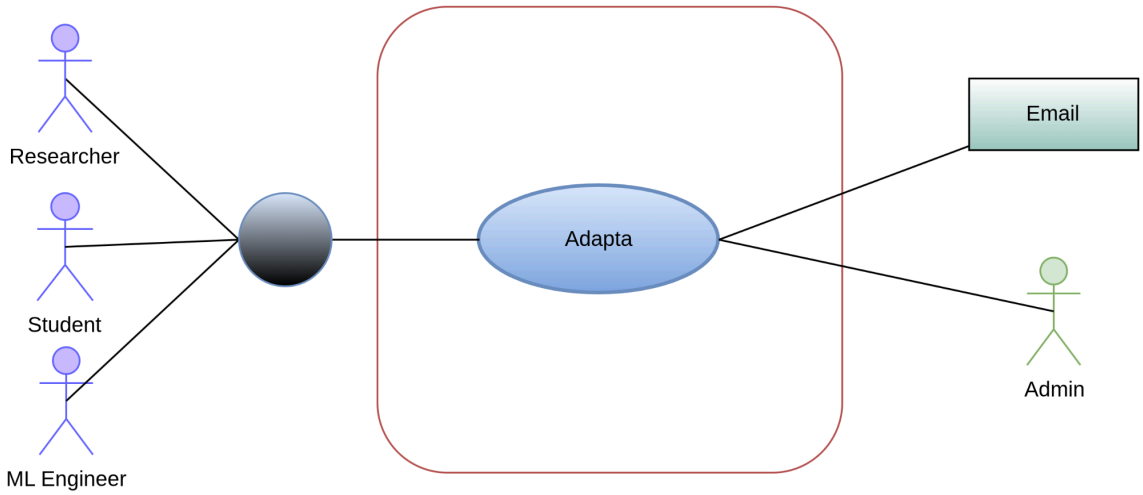


Figure 1: Level 0 use case diagram of Adapta

At the highest level of abstraction, the framework accepts a source dataset and a target dataset from the user, performs domain adaptation using a selected method, trains and evaluates a classifier on the adapted representation, and returns evaluation metrics, visualizations, and an exportable report. The user configures the experiment; the framework carries it out and reports the outcome.

Level 1
Use Case ID: 1
Name: Adapta: General Purpose Transfer Learning Framework - detailed
Primary Actor: Researcher, Student, Teacher, ML Engineer, System
Secondary Actor: Email, Admin

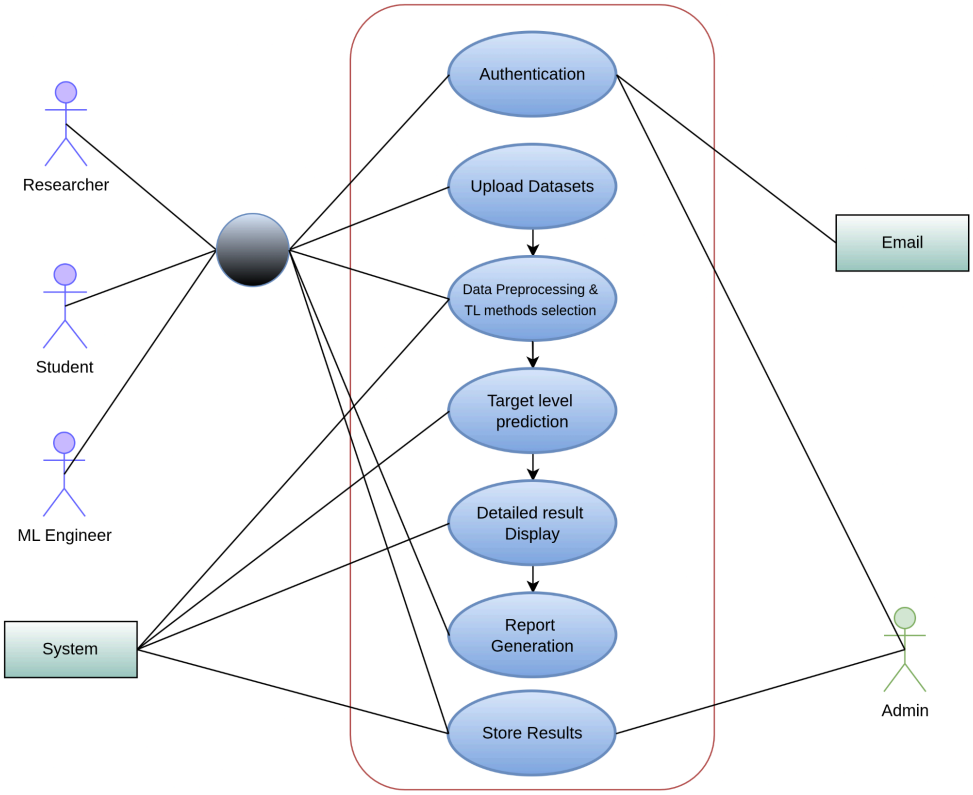


Figure 2: Level 1 use case diagram of Adapta

Decomposing the system reveals seven principal use cases, corresponding to the seven functional modules of the framework.

3.1.2.1 Authentication

Functionality: The user logs into the framework using valid credentials. The framework authenticates the user, establishes a secure session, and grants access according to the assigned role (Researcher, Student, ML Engineer, or Administrator). Authentication failures are reported with appropriate error messages. Administrative users may additionally manage user accounts and permissions.

Purpose: To ensure that only authorized users can access the framework and its resources while protecting experimental data and stored results.

Key role: Acts as the security gateway of the framework by validating user identity before any experiment can be performed.

Actors involved: Researcher, Student, Teacher, ML Engineer, Email, Admin

3.1.2.2 Upload Datasets

Functionality: The user uploads the source and target datasets from local storage. The framework supports multiple formats (e.g., CSV, MAT, and ARFF), validates file integrity, verifies dataset compatibility, detects missing values and inconsistent labels, extracts metadata such as the number of samples, features, classes, and data types, and assigns source and target domain roles.

Purpose: To ensure that only valid and compatible datasets are used for transfer learning experiments.

Key role: Serves as the entry point for all experimental workflows by importing and validating datasets.

Actors involved: Researcher, Student, Teacher, ML Engineer

3.1.2.3 Data Preprocessing & Transfer Learning Method Selection

Functionality: The framework preprocesses the uploaded datasets using the selected normalization or standardization technique. The user simultaneously selects a transfer learning method (e.g., TCA, CORAL, JDA, BDA, OCMTL, or OCMTL+) and configures its parameters. The framework validates parameter values and prepares both datasets for domain adaptation.

Purpose: To transform datasets into a suitable representation while allowing users to configure the desired transfer learning algorithm.

Key role: Acts as the experiment configuration and data preparation module, ensuring consistent preprocessing and proper method selection.

Actors involved: Researcher, Student, Teacher, ML Engineer

3.1.2.4 Target Level Prediction

Functionality: The framework executes the selected transfer learning algorithm, learns a shared feature representation between the source and target domains, trains the selected classifier using the transformed source data, and predicts labels for the target dataset. Throughout execution, progress information is displayed, and computational errors are handled gracefully.

Purpose: To perform cross-domain classification by transferring knowledge from the labeled source domain to the unlabeled or sparsely labeled target domain.

Key role: Represents the core computational module where domain adaptation and target prediction are performed.

Actors involved: Researcher, Student, Teacher, ML Engineer, System

3.1.2.5 Detailed Result Display

Functionality: After prediction, the framework displays comprehensive experimental results, including classification metrics such as Accuracy, Precision, Recall, F1-score, AUC, confusion matrix, execution time, and comparisons with baseline models. The interface may additionally present visualizations of domain adaptation and feature distributions.

Purpose: To provide users with a comprehensive interpretation of experimental performance.

Key role: Acts as the primary visualization and analysis interface for completed experiments.

Actors involved: Researcher, Student, Teacher, ML Engineer, System

3.1.2.6 Report Generation

Functionality: The framework automatically generates an experiment report containing dataset information, preprocessing settings, selected transfer learning method, classifier configuration, parameter values, evaluation metrics, visualizations, and experimental conclusions. Reports can be exported in formats such as PDF or CSV.

Purpose: To facilitate documentation, reproducibility, and sharing of experimental outcomes.

Key role: Produces standardized reports that summarize the complete experimental workflow.

Actors involved: Researcher, Student, Teacher, ML Engineer

3.1.2.7 Store Results

Functionality: The framework stores experiment configurations, evaluation metrics, prediction results, generated reports, and associated metadata in persistent storage. Saved experiments can later be retrieved, compared, or reused by authorized users. Administrators oversee storage management and data integrity.

Purpose: To preserve experimental history and support reproducibility, comparison, and future analysis.

Key role: Acts as the persistence module responsible for maintaining experiment records.

Actors involved: Researcher, Student, Teacher, ML Engineer, System

Level 1.1
Use Case ID: 1.1
Name: Authentication
Primary Actor: Researcher, Student, Teacher, ML Engineer
Secondary Actor: Email, Admin

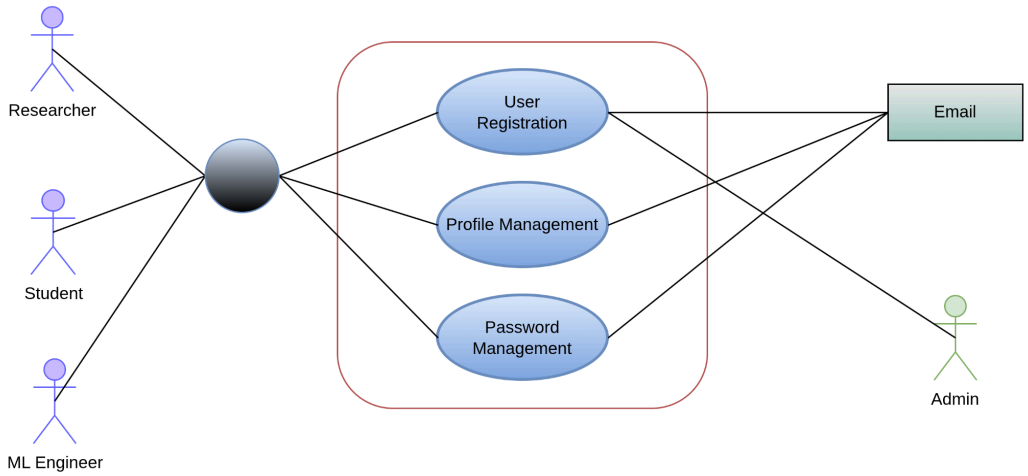


Figure 3: Level 1.1 (Authentication) use case diagram of Adapta

Level 1.2
Use Case ID: 1.2
Name: Upload Dataset
Primary Actor: Researcher, Student, ML Engineer

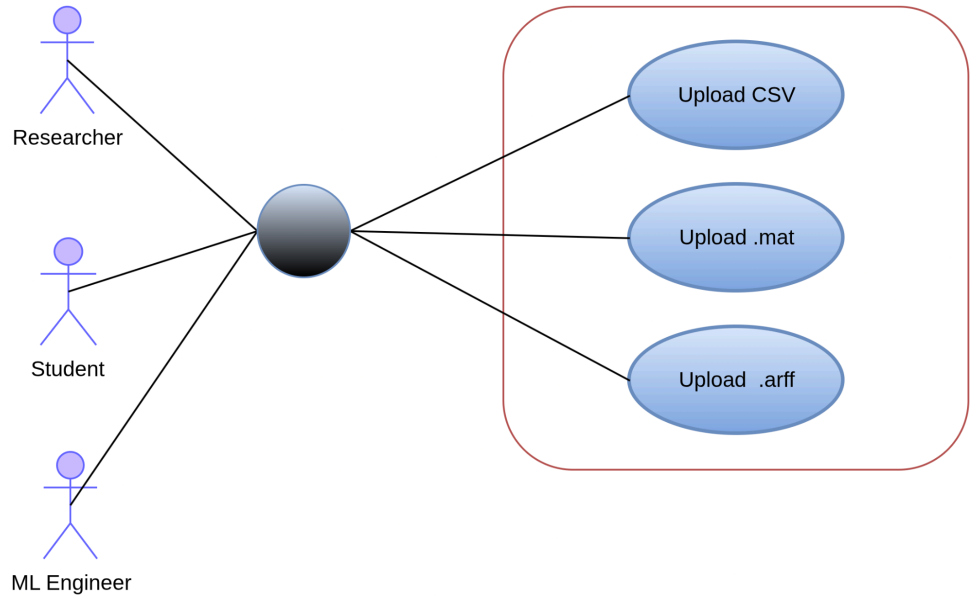


Figure 4: Level 1.2 (Upload Dataset) use case diagram of Adapta

Level 1.3
Use Case ID: 1.3
Name: Data Preprocessing
Primary Actor: Researcher, Student, ML Engineer
Secondary Actor: System

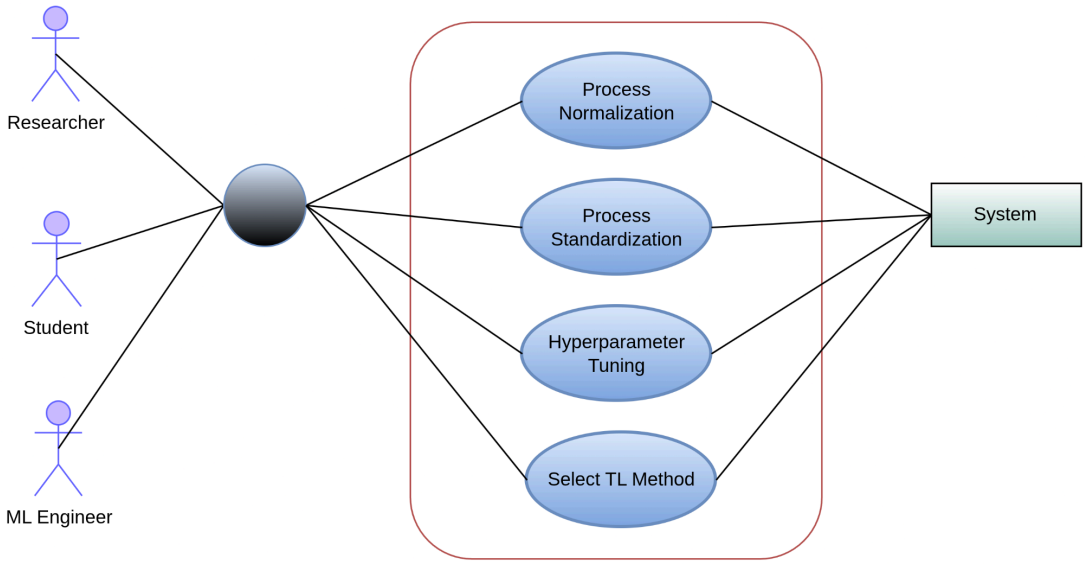


Figure 5: Level 1.3 (Data Preprocessing) use case diagram of Adapta

Level 1.4
Use Case ID: 1.4
Name: Label Prediction
Primary Actor: Researcher, Student, ML Engineer, System

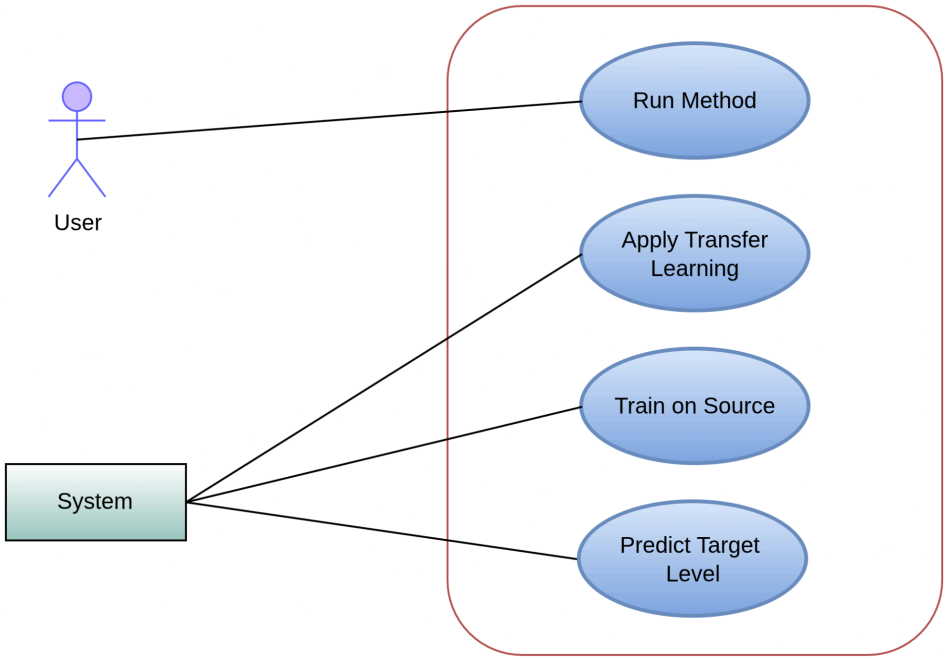
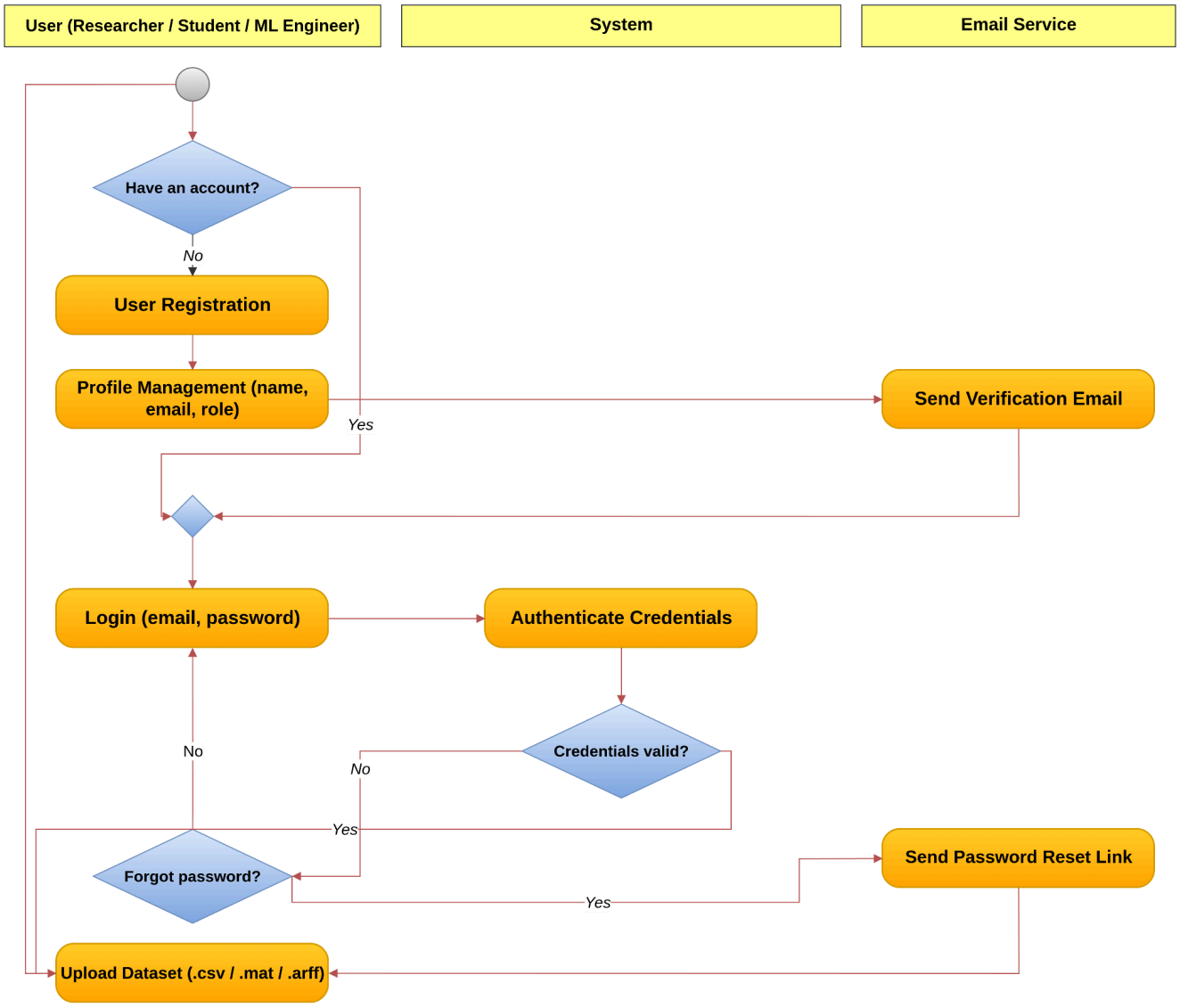


Figure 6: Level 1.4 (Label Prediction) use case diagram of Adapta

3.2 Swimlane Diagram

The swimlane diagram below models the control flow of a complete cross-domain classification experiment, from the moment the user opens the framework to the moment results are exported. It captures the decision points at which the framework may return control to the user, namely dataset validation failure, feature dimension mismatch, invalid hyperparameters, and numerical failure during adaptation.



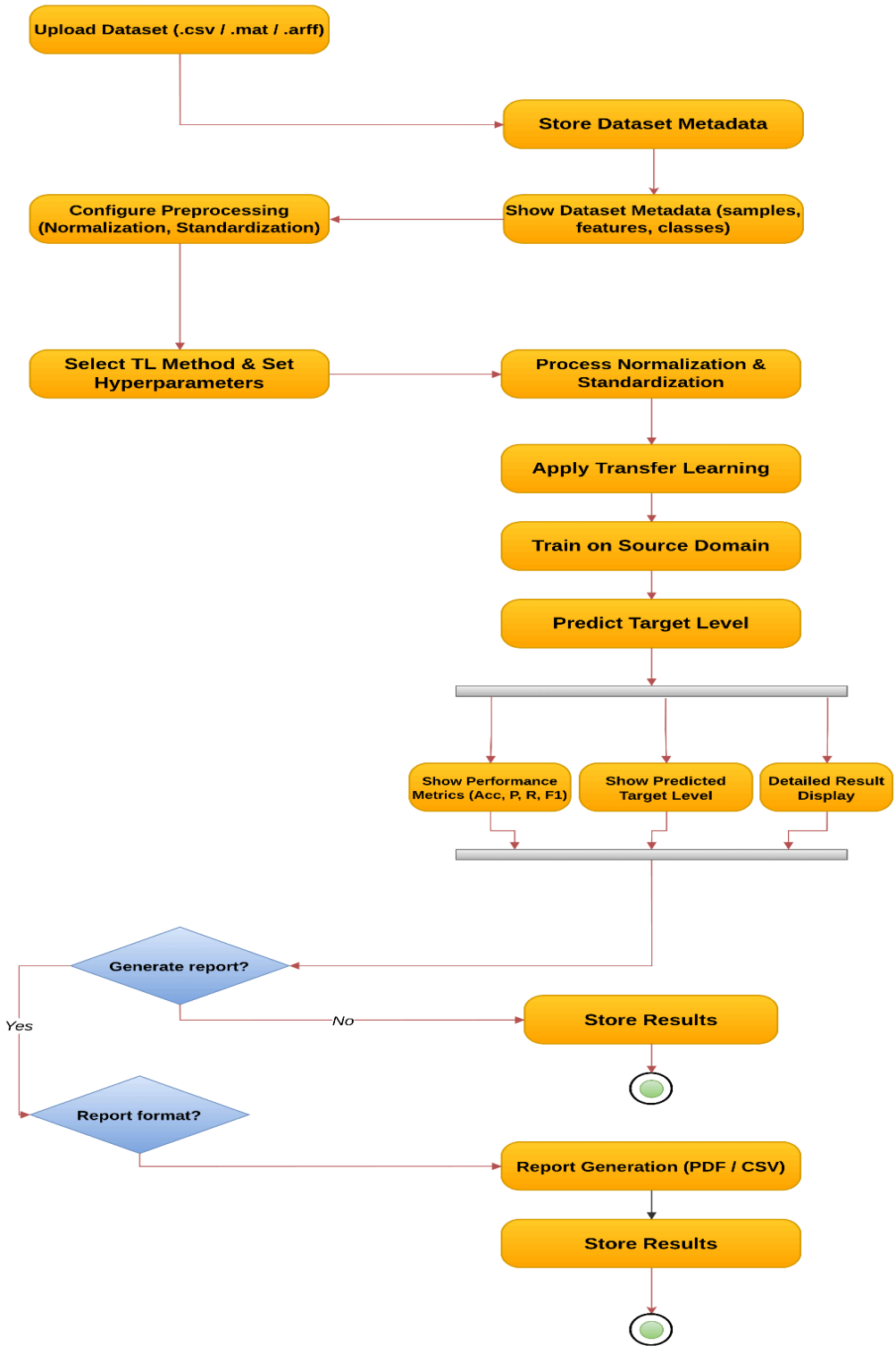


Figure 7: Swimlane diagram of Adapta

4. Data Based Modeling

This framework is an AI-based research tool rather than a transactional information system. It does not maintain a relational database of business entities; its persistent state consists of experiment configurations, cached results, and exported reports stored on the local file system in structured form. Accordingly, this chapter presents the dataset description required for an AI-based project, together with the identification of the data objects that the framework manipulates internally and the flow of data between its modules.

4.1 Data Object Identification

Applying the standard selection criteria — retained information, needed services, multiple attributes, common attributes and operations, and essential requirements — to the nouns of the problem statement yields the following persistent or semi-persistent data objects. These are the entities whose state the framework must retain across the stages of an experiment.

Data Object	Attributes	Description
User	user_id, name, email, role (researcher / student/ ML engineer)	Stores registered user information. **PK:** user_id.
Experiment	experiment_id, user_id, source_dataset_id, kernel_type, target_dataset_id, latent_feature, algorithm_name, classifier_type, normalization_type, status, standardization_type, date	Stores the configuration and execution details of a transfer learning experiment. **PK:** experiment_id. **FK:** user_id → User(user_id). dataset_id →(dataset_id)
Dataset	dataset_id, name, file_format, n_samples, n_features, feature_name, n_classes, class_name	Stores uploaded dataset information and extracted metadata. **PK:** dataset_id.
Evaluation	result_id, experiment_id, accuracy, precision, recall, f1_score	Stores the evaluation metrics of an experiment. **PK:** result_id. **FK:** experiment_id → Experiment(experiment_id).

Table 3: Attributes of the principal data objects

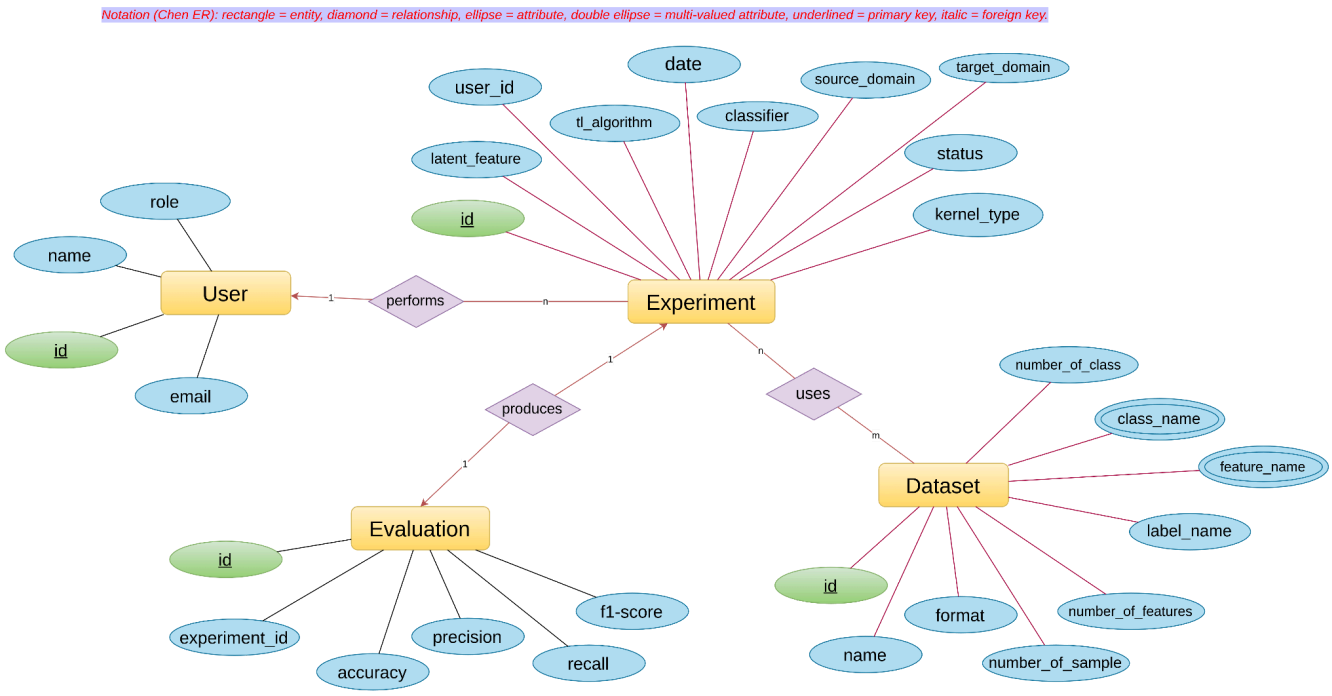


Figure 8: Entity Relation Diagram of Adapta

The relationships among these objects are as follows. A User may conduct many Experiments, while each Experiment belongs to exactly one User. A Dataset may be used in many Experiments, and each Experiment references one source dataset and one target dataset. Each Experiment produces exactly one Evaluation, which stores the performance metrics obtained after execution. This structure ensures that experiments can be reproduced, datasets can be reused across multiple experiments, and evaluation results remain directly associated with the corresponding experiment.

5. Class-Based Modeling

Class-based modeling describes the structure of the system in terms of the classes it comprises, their attributes and operations, and the collaborations among them. It bridges the analysis expressed in the preceding chapters and the architectural design of the next.

5.1 Analysis Classes

Nouns were extracted from the project proposal, the user story, and the requirements and were then filtered by general classification — external entities, things, events, roles, organizational units, places, and structures — and by the standard selection criteria of retained information, needed services, multiple attributes, common attributes, common operations, and essential requirements. The following analysis classes were identified:

1. DatasetManager — loads, validates, and describes datasets, and manages source and target domain assignment.
2. PreprocessingEngine — applies normalization, standardization, and feature preparation consistently to both domains.
3. TransferMethod — the abstract interface implemented by every adaptation method, defining fit and transform behaviour.
4. TransferMethodRegistry — maintains the catalogue of available methods and instantiates them by name, providing the extension point for new methods.
5. ClassifierTrainer — trains the selected baseline classifier on adapted features and produces predictions for the target domain.
6. EvaluationEngine — computes the classification metric suite and the no-transfer baseline comparison.
7. VisualizationEngine — produces PCA and t-SNE projections, alignment scatter plots, and comparison charts.
8. ExperimentManager — orchestrates the pipeline, holds the experiment configuration, and manages saving and reloading.
9. ReportExporter — assembles and writes PDF and CSV reports.

5.2 Class Cards

The responsibilities, attributes, operations, and collaborators of each analysis class are recorded below.

DatasetManager	
Attributes	**Methods**
- datasets - source_domain - target_domain - supported_formats - validation_errors	+ load_dataset(path, format) + validate_dataset(data) + extract_properties(data) + assign_source_domain(id) + assign_target_domain(id) + check_dimension_compatibility() + get_dataset_summary(id)
Responsibilities	**Collaborators**
Load datasets from .csv, .mat and .arff files, validate their structure, extract descriptive metadata, and manage the assignment of source and target roles. Reject incompatible domain pairs before any computation begins.	• PreprocessingEngine • ExperimentManager • TransferMethod

Table 4: Class Card (DatasetManager)

PreprocessingEngine	
Attributes	**Methods**
- normalization_type - standardization_type - label - features	+ normalize(X) + standardize(X) + extract_label() + get_config()
Responsibilities	**Collaborators**
Apply the user-selected scaling and preparation operations identically to the source and target domains, and retain the fitted statistics so that preprocessing is reproducible and comparable across methods.	• DatasetManager • TransferMethod • ExperimentManager

Table 5: Class Card (PreprocessingEngine)

TransferMethod (abstract; implemented by each Transfer Learning method)	
Attributes	**Methods**
- method_name - latent_dim - kernel_type - kernel_param - regularization_lambda - balance_factor_mu - n_iterations - transformation_matrix	+ fit(X_source, X_target, y_source) + transform(X) + fit_transform(...) + compute_kernel(X1, X2) + get_transformation() + get_hyperparameter_schema() + validate_params()
Responsibilities	**Collaborators**
Define the uniform contract by which every adaptation method is invoked. Compute the transformation that projects both domains into a shared latent space in which distributional discrepancy is reduced, and expose the hyperparameters it accepts so that the interface can render them.	• PreprocessingEngine • TransferMethodRegistry • ClassifierTrainer

Table 6: Class Card (TransferMethod)

ClassifierTrainer	
Attributes	**Methods**
- classifier_name - classifier_params - trained_model - predictions - training_time	+ build_classifier(name, params) + train(X_source, y_source) + predict(X_target) + predict_proba(X_target) + train_baseline(X_raw, y) + get_model()
Responsibilities	**Collaborators**
Instantiate and train the selected baseline classifier on the adapted source representation, produce predictions and class probabilities for the target domain, and additionally train the no-transfer baseline required for negative transfer detection.	• TransferMethod • EvaluationEngine • ExperimentManager

Table 7: Class Card (ClassifierTrainer)

EvaluationEngine	
Attributes	**Methods**
- accuracy - precision - recall - f1_score - auc - confusion_matrix - baseline_accuracy	+ compute_accuracy(y_true, y_pred) + compute_precision() + compute_recall + compute_f1() + compute_auc(y_true, y_proba) + build_confusion_matrix() + compare_with_baseline() + aggregate_results() + get_metric_table()
Responsibilities	**Collaborators**
Compute the complete metric suite on the target domain, assemble it into a comparable record, and evaluate the adapted result against the no-transfer baseline.	• ClassifierTrainer • ReportExporter

Table 8: Class Card (EvaluationEngine)

VisualizationEngine	
Attributes	**Methods**
- projection_method - n_components - color_map - figure_cache	+ project_pca(X, n_components) + project_tsne(X, perplexity) + plot_before_after(X_src, X_tgt) + plot_domain_alignment() + plot_metric_comparison(results) + plot_confusion_matrix() + save_figure(path)
Responsibilities	**Collaborators**
Reduce high-dimensional feature spaces to two dimensions and render source and target distributions before and after adaptation, together with comparison charts, so that the effect of adaptation is directly observable.	• EvaluationEngine • ReportExporter • ExperimentManager

Table 9: Class Card (VisualizationEngine)

ExperimentManager	
Attributes	**Methods**
- experiment_id - configuration - status - date - results - experiment_history	+ create_experiment(config) + run_experiment() + run_comparative_batch(methods) + cancel_experiment() + save_configuration(name) + load_configuration(name) + get_dashboard_summary()
Responsibilities	**Collaborators**
Orchestrate the full pipeline from preprocessing through export, hold the configuration that defines a run, report progress to the interface, and manage persistence and reload of configurations for reproducibility.	• DatasetManager • PreprocessingEngine • TransferMethodRegistry • ClassifierTrainer • EvaluationEngine • ReportExporter

Table 10: Class Card (ExperimentManager)

ReportExporter	
Attributes	**Methods**
- export_format - output_path - included_sections - generated_at - experimented_by	+ export_pdf(experiment) + export_csv(results) + embed_figures(figures) + export_comparison_table() + get_export_status()
Responsibilities	**Collaborators**
Assemble the configuration, metric tables, analysis, and figures of one or more experiments into a self-contained PDF report or a machine-readable CSV file for downstream analysis.	• EvaluationEngine • VisualizationEngine • ExperimentManager

Table 11: Class Card (ReportExporter)

5.3 Class Responsibility and Collaboration Diagram

The CRC diagram below summarizes the collaborations recorded in the class cards. The ExperimentManager sits at the centre of the model, coordinating the linear chain from DatasetManager through PreprocessingEngine, TransferMethodRegistry and its TransferMethod implementations, ClassifierTrainer, and EvaluationEngine, while DomainAnalyzer and VisualizationEngine attach as analytical branches and ReportExporter consumes the assembled outcome.

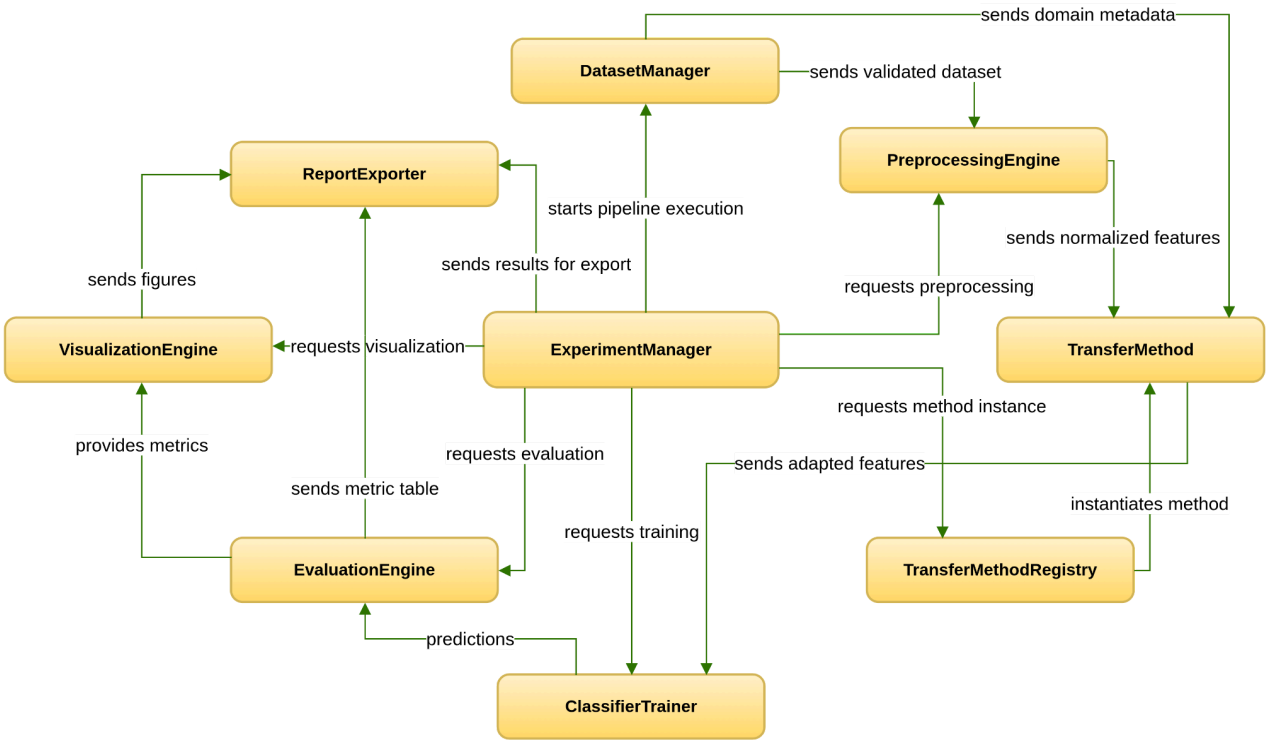


Figure 9: Class responsibility and collaboration (CRC) diagram

6. Software Design and Architectural Design

The architectural design defines the high-level structure of the framework: its components, the responsibilities allocated to each, the interfaces across which they communicate, and the way data moves between them. The architecture is layered, which keeps numerical computation independent of presentation and allows the adaptation methods to be extended without disturbing the rest of the system.

6.1 Architectural Context Diagram

An Architectural Context Diagram (ACD) is a high-level diagram that shows the system as a single unit and its interactions with external entities, such as users, external systems, databases, APIs, or services. It defines the boundary of the system and what lies outside it. It answers the question: "What external actors and systems interact with our system?"

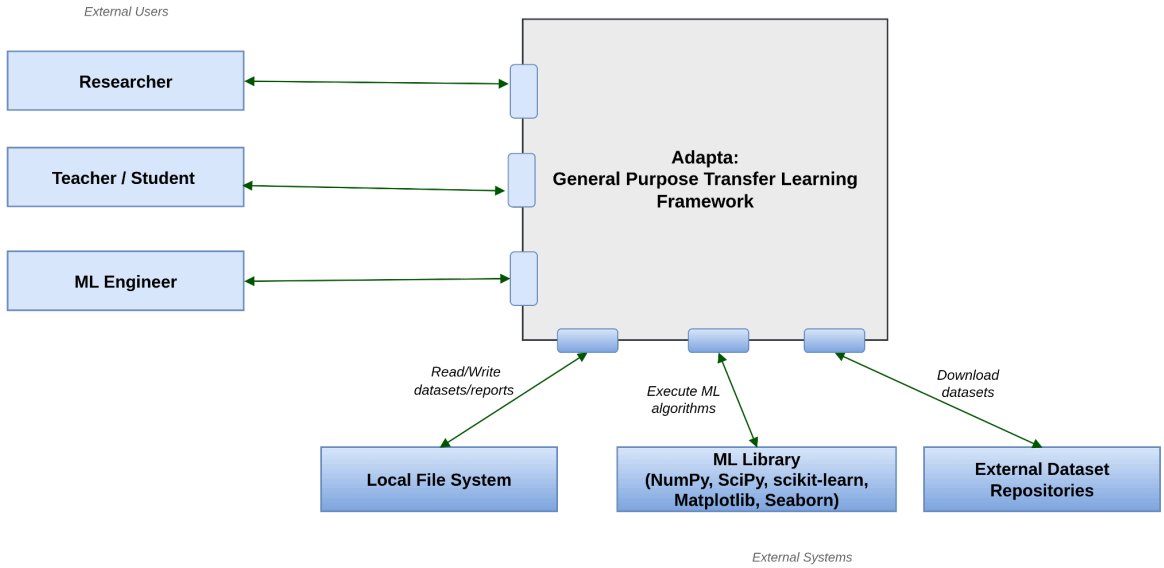


Figure 10: Architectural context diagram

The architectural context diagram positions the framework with respect to the actors and systems around it. Researchers, Teachers, and Students act as external users, supplying datasets and experiment configurations and consuming metrics, visualizations, and reports. The Local File System acts as both an external and a peer system: it supplies dataset files and receives saved configurations and exported reports. ML Library — NumPy, SciPy, scikit-learn, Matplotlib and Seaborn — acts as an external system performing the linear algebra, classification, and rendering on which the framework depends. External Dataset Repositories act as peers from which benchmark datasets are obtained prior to use, outside the running system.

6.3 Top-Level Components

The component diagram below shows the top-level components of the framework and the interfaces across which they interact. Each component corresponds to one coherent responsibility identified in the class-based model, and the dependencies among them are one-directional from presentation towards domain logic, so that no numerical module depends on the interface.

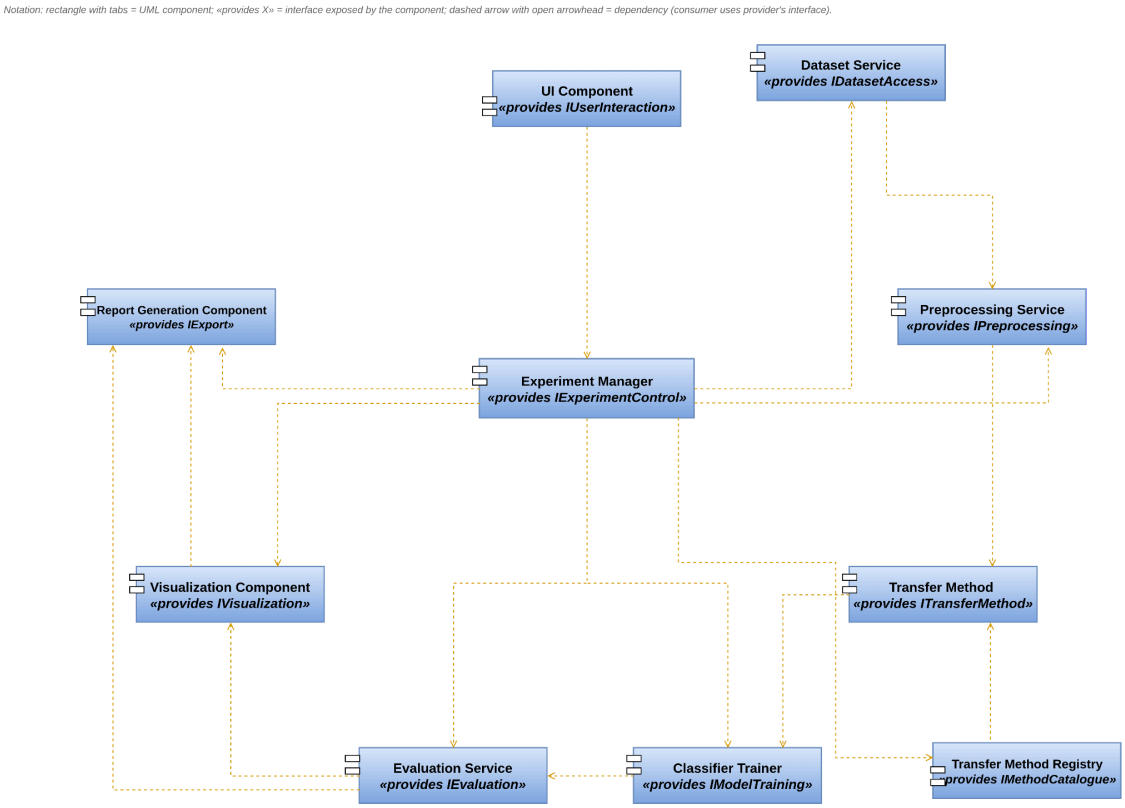


Figure 11: Top-level component diagram

6.4 Mapping Requirements to Software Architecture

This section establishes traceability between the requirements and the architectural components defined above, so that every stated requirement can be shown to be supported by an identified part of the design.

Req. ID	Requirement	Mapped Architectural Components
FR-1	Import datasets in .csv, .mat and .arff formats	Dataset Service, File Adapters, UI Component
FR-2	Validate datasets and report structural problems	Dataset Service, UI Component
FR-3	Display dataset properties (samples, features, classes, types)	Dataset Service, UI Component

Req. ID	Requirement	Mapped Architectural Components
FR-4	Assign source and target domains and check compatibility	Dataset Service, Experiment Manager, UI Component
FR-5	Apply configurable preprocessing to both domains	Preprocessing Service, Experiment Manager
FR-6	Select a transfer learning method from the supported set	Method Registry, Transfer Method, UI Component
FR-7	Configure method hyperparameters	Method Registry, UI Component
FR-8	Select and train a baseline classifier	Classifier Trainer, Experiment Manager
FR-9	Execute the complete adaptation and training pipeline	Experiment Manager, Transfer Method, Classifier Trainer
FR-10	Evaluate on the target domain with the full metric suite	Evaluation Service, Classifier Trainer
FR-11	Compute MMD before and after adaptation	Transfer Method
FR-12	Visualize feature distributions with PCA and t-SNE	Visualization Component, UI Component
FR-13	Compare methods and rank them on the same configuration	Experiment Manager, Evaluation Service, Visualization Component
FR-14	Detect and report negative transfer	Evaluation Service, Classifier Trainer
FR-16	Save and reload experiment configurations	Configuration Store, Experiment Manager
FR-17	Export results and figures as PDF and CSV	Report Exporter, Visualization Component
FR-18	Present an experiment dashboard	Experiment Manager, UI Component
FR-20	Support extension with new adaptation methods	Method Registry, ITransferMethod interface
NFR-1	Performance within stated bounds	Execution Engine, Transfer Method
NFR-3	Graceful error handling	Execution Engine, UI Component
NFR-4	Reproducibility from saved configuration and seed	Experiment Manager
NFR-5	Modularity across layers	All components via declared interfaces

Table 12: Requirement traceability matrix

7. Preliminary Test Plan

This chapter presents the testing objectives for the framework and the features that will be tested, followed by the preliminary test cases. The purpose of the test plan is to establish that the framework computes correct results, behaves predictably under invalid input, and reproduces the published behaviour of the methods it implements.

7.1 Testing Objectives

- Ensure correct parsing of all supported dataset formats and accurate metadata extraction.
- Validate proper handling of invalid datasets and input errors.
- Confirm the correct application of preprocessing techniques.
- Evaluate the correctness of the implemented transfer learning methods.
- Assess classifier training and target-domain prediction accuracy.
- Validate the computation of evaluation metrics, including Accuracy, Precision, Recall, F1-score, and AUC.
- Confirm correct computation of domain adaptation measures and negative transfer detection.
- Ensure accurate generation of visualizations and result displays.
- Validate experiment reproducibility using saved configurations.
- Confirm successful generation of PDF and CSV reports.
- Evaluate system responsiveness during long-running experiments.
- Assess system performance on large-scale datasets.

7.2 Test Cases

The preliminary test cases for the framework are documented below.

Test Case ID: T1

Field	Description
Title	Dataset Import and Property Extraction
Test Scenario	Import an Office-Caltech SURF dataset in .mat format and a RELINK dataset in .csv format through the dataset panel.
Expected Outcome	Both files are parsed without error, and the reported sample count, feature count, class count, class distribution and data types match the actual contents of each file.

Table 13: Test Case T1

Test Case ID: T2

Field	Description
Title	Invalid Dataset Rejection
Test Scenario	Import a .csv file containing a row with a missing feature value and a second file with no label column.
Expected Outcome	The framework reports the specific defect and its location for each file, refuses to make the dataset available for domain assignment, and remains operational.

Table 14: Test Case T2

Test Case ID: T3

Field	Description
Title	Incompatible Domain Pair Detection
Test Scenario	Assign an 800-dimensional SURF dataset as source and a 4096-dimensional DeCAF dataset as target, then attempt to start an experiment.
Expected Outcome	The framework detects the dimensionality mismatch at assignment time, explains the problem, and prevents the experiment from being started.

Table 15: Test Case T3

Test Case ID: T4

Field	Description
Title	Preprocessing Correctness and Consistency
Test Scenario	Apply z-score standardization to a source and target pair and inspect the resulting matrices.
Expected Outcome	Each standardized feature has zero mean and unit variance under the selected fitting policy, and the identical transformation parameters are applied to both domains.

Table 16: Test Case T4

Test Case ID: T5

Field	Description
Title	Adaptation Method Numerical Correctness
Test Scenario	Run TCA, JDA, BDA and CORAL on the COIL20 benchmark using the standard protocol with a 1-nearest-neighbour classifier, for all two domain pairs.
Expected Outcome	The reported target accuracy for each method and domain pair falls within the documented tolerance of the accuracy published in the corresponding original paper.

Table 17: Test Case T5

Test Case ID: T6

Field	Description
Title	Hyperparameter Validation
Test Scenario	Set the latent dimension to zero, to a value exceeding the feature count, and to a non-integer value; set the regularization coefficient to a negative value.
Expected Outcome	Each invalid value is rejected at entry with an explanatory message, and the experiment cannot be started until all values are valid.

Table 18: Test Case T6

Test Case ID: T7

Field	Description
Title	Evaluation Metric Correctness
Test Scenario	Run an experiment on a dataset with a known confusion matrix and compare the reported metrics against values computed independently with scikit-learn.
Expected Outcome	Accuracy, Precision, Recall, F1-score and AUC agree with the independently computed reference values to within floating-point tolerance.

Table 19: Test Case T7

Test Case ID: T8

Field	Description
Title	MMD Reduction After Adaptation
Test Scenario	Run TCA on a domain pair known to benefit from adaptation and record MMD before and after the transformation.
Expected Outcome	MMD after adaptation is lower than MMD before adaptation, and repeating the run with the same seed yields an identical value.

Table 20: Test Case T8

Test Case ID: T9

Field	Description
Title	Negative Transfer Detection
Test Scenario	Run an experiment on a deliberately unrelated domain pair, for example a text source domain against an image target domain of matching dimensionality, and compare the adapted result against the no-transfer baseline.
Expected Outcome	Adapted accuracy falls below the baseline, the experiment is explicitly flagged as negative transfer, and the reported domain similarity score is correspondingly low.

Table 21: Test Case T9

Test Case ID: T10

Field	Description
Title	Feature Space Visualization
Test Scenario	Generate PCA and t-SNE projections of a domain pair before and after adaptation and inspect the rendered plots.
Expected Outcome	Four correctly labelled plots are produced, source and target points are visually distinguishable, and the domain clouds overlap more after adaptation than before.

Table 22: Test Case T10

Test Case ID: T11

Field	Description
Title	Comparative Analysis and Ranking
Test Scenario	Execute a comparative batch over all supported methods on a single domain pair with a fixed configuration.
Expected Outcome	A single table is produced containing one row per method plus the no-transfer baseline, ranked by target accuracy, with any negative transfer cases marked.

Table 23: Test Case T11

Test Case ID: T12

Field	Description
Title	Configuration Persistence and Reproducibility
Test Scenario	Save the configuration of a completed experiment, restart the application, reload the configuration and re-execute it with the same random seed.
Expected Outcome	The reloaded configuration is identical in every field, and the re-executed run reproduces the previously reported metrics exactly for deterministic methods and within the stated tolerance for stochastic components.

Table 24: Test Case T12

8. Timeline

The development of the framework is organized into 14 phases spanning the project period. The Gantt chart below records the planned schedule; shaded cells indicate the weeks in which each activity is active.

Phase / Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14
1. Literature review and method study														
2. Requirement analysis and system modeling														
3. Architectural and detailed design														
4. Data management and preprocessing module														
5. Implementation of baseline methods (TCA, JDA, BDA, CORAL)														
6. User interface integration														
6. Implementation of methods (GFK, JCSA, DALI, OCMTL)														
7. Design and implementation of proposed method														
8. Evaluation, visualization and analyzer modules														
10. Testing, benchmarking and validation														
11. Documentation and final report														

Table 25: Project timeline (Gantt chart)

9. Conclusion

This report has presented the requirement analysis, system modeling, data and information modeling, software design, architectural design, AI engineering design, and preliminary test plan for the General Purpose Transfer Learning Framework for Cross-Domain Classification. The framework addresses a concrete and recurring difficulty in applied machine learning: that target domains are frequently label-scarce and distributionally different from the source data available, that many adaptation methods exist to correct this shift, and that comparing them fairly is at present far more laborious than it should be.

The design responds to this by unifying the experimental pipeline behind one interface and by placing every adaptation method behind a common contract, so that methods become interchangeable and comparisons become like-for-like. The layered architecture separates numerical computation from presentation and confines extension to a single interface, which keeps the cost of adding a new method low. The analytical modules go beyond executing experiments to explaining them, quantifying domain relatedness through MMD and distribution overlap, flagging negative transfer against an explicit no-transfer baseline, and rendering the effect of adaptation visible through PCA and t-SNE projections of the feature space.

The proposed method constitutes the research contribution of the project. By incorporating a measured domain similarity signal into the core-extraction step of OCMTL, it aims to modulate the degree of transfer according to how related the domains actually are, and thereby to reduce negative transfer in weakly related pairs. Because the framework enforces an identical protocol across all methods, the evaluation of against the eight established methods will be directly comparable and reproducible.

Beyond its research use, the framework is intended as a teaching instrument. A student can observe distribution shifts, apply an adaptation method, and see the two domain clouds move towards one another without first having to implement any of the underlying mathematics, and a teacher can demonstrate negative transfer as a live result rather than as an abstract caution.

The remaining work consists of completing the implementation of the later adaptation methods, finalizing and evaluating, integrating the analyzer and visualization modules with the interface, and carrying out the benchmarking and validation described in the test plan. On completion, the framework is expected to provide a reusable, extensible, and reproducible platform for cross-domain classification research and instruction.

References

[1] S. J. Pan, I. W. Tsang, J. T. Kwok, and Q. Yang, "Domain adaptation via transfer component analysis," IEEE Transactions on Neural Networks, vol. 22, no. 2, pp. 199–210, 2011.

[2] M. Long, J. Wang, G. Ding, J. Sun, and P. S. Yu, "Transfer feature learning with joint distribution adaptation," in Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2013, pp. 2200–2207.

[3] J. Wang, Y. Chen, S. Hao, W. Feng, and Z. Shen, "Balanced distribution adaptation for transfer learning," in Proceedings of the IEEE International Conference on Data Mining (ICDM), 2017, pp. 1129–1134.

[4] S. S. Nahid, et al., "OCMTL: Transfer learning by orthogonal core-extraction of a matrix and its application in cross-project defect prediction," Alexandria Engineering Journal, vol. 133, pp. 498–516, 2025.

[5] A. K. Devika, et al., "Visual domain adaptation through locality information," Engineering Applications of Artificial Intelligence, vol. 123, art. 106172, 2023.

[6] B. Sun, J. Feng, and K. Saenko, "Correlation alignment for unsupervised domain adaptation," in Domain Adaptation in Computer Vision Applications, Cham: Springer, 2017, pp. 153–171.

[7] B. Gong, Y. Shi, F. Sha, and K. Grauman, "Geodesic flow kernel for unsupervised domain adaptation," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2012, pp. 2066–2073.

[8] S. J. Pan and Q. Yang, "A survey on transfer learning," IEEE Transactions on Knowledge and Data Engineering, vol. 22, no. 10, pp. 1345–1359, 2010.

[9] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," Journal of Big Data, vol. 3, no. 9, pp. 1–40, 2016.

[10] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, "A kernel two-sample test," Journal of Machine Learning Research, vol. 13, pp. 723–773, 2012.

[11] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," Journal of Machine Learning Research, vol. 9, pp. 2579–2605, 2008.

[12] F. Pedregosa, et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[13] Riverbank Computing, "PyQt documentation," [Online]. Available: https://www.riverbankcomputing.com/static/Docs/PyQt6/